

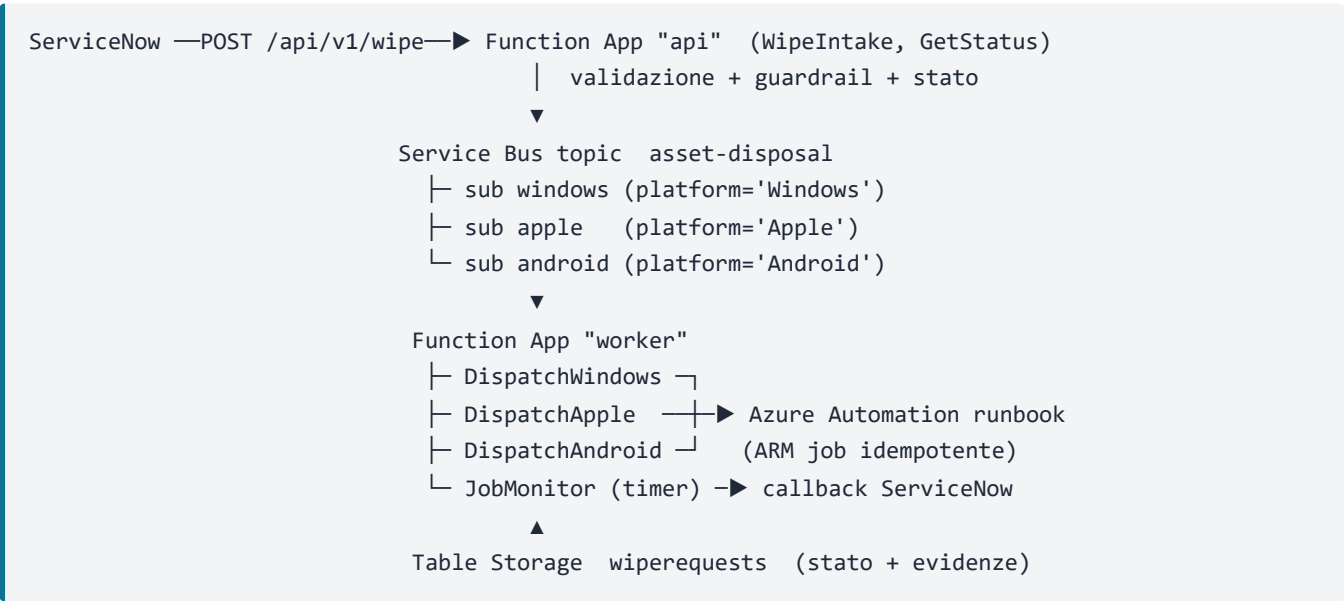
README

PoC PowerShell — Dispatch su Service Bus + runbook di piattaforma

Evoluzione di `poc-powershell-mock`: la Function **non esegue più il wipe**. Riceve la richiesta da ServiceNow, la valida, la mette su un **topic Service Bus** e un **worker di backend** instrada la richiesta verso il **runbook Azure Automation specifico per piattaforma** già in uso presso il cliente.

Il razionale, le alternative valutate e i gap dei runbook attuali sono documentati in `docs/evoluzione-dispatch-runbook.md`.

Architettura



Perché due Function App

`api` e `worker` girano sullo **stesso App Service Plan** (nessun costo aggiuntivo) ma hanno **identità gestite distinte**:

App	Trigger	Permessi
api	HTTP (pubblico)	Graph <i>read-only</i> , Service Bus Sender , Table
worker	Service Bus + timer (nessun endpoint pubblico)	Service Bus Receiver , Automation Job Operator , Table

L'app esposta su internet non ha quindi alcun permesso per far partire un wipe: può solo accodare una richiesta.

Contratto REST

POST /api/v1/wipe

```
{
  "requestId": "SNOW-RITM0012345",    // opzionale: chiave di idempotenza
  "operatingSystem": "Windows",        // Windows | macOS | iOS | Android | Mobile
  "scenario": "Disposal",              // Retirement | Sale | Disposal | LostStolen
  "serialNumber": "5CG1234ABC",
  "imei": "358240051111110",          // opzionale (Android/KME)
  "deviceName": "FC1WRK001",          // opzionale
  "managedDeviceId": "...",            // opzionale
  "mdmServerId": "...",               // opzionale (Apple ABM)
  "userConfirmed": true,               // guardrail di processo
  "callbackUrl": "https://.../callback",
  "dryRun": false
}
```

Risposte:

Codice	Significato
202	Richiesta accodata. Header Location: /api/v1/wipe/status?requestId=...
200	requestId già visto → stato corrente, duplicate: true
400	Payload non valido
422	status: Rejected + reason (DeviceNotManagedByIntune, GuardrailFailed, AmbiguousPlatform) → ServiceNow apre un task manuale
502	Graph non raggiungibile

operatingSystem: "Mobile" è ambiguo: l'intake risolve iOS vs Android interrogando managedDevices.operatingSystem in Intune prima di instradare.

GET /api/v1/wipe/status?requestId=...

Restituisce lo stato dalla macchina a stati:

```
Accepted → Queued → Dispatching → Dispatched → Running → Completed
                                         \ PartiallyCompleted
                                         \ Failed
Rejected (guardrail)  DispatchFailed (runbook non avviabile)
```

`PartiallyCompleted` copre il caso "unenrollment riuscito ma wipe fallito", che il processo deve poter distinguere da un successo pieno.

Meccanismo di dispatch

Il worker crea il job con `PUT .../automationAccounts/{aa}/jobs/{jobName}?api-version=2023-11-01` usando la managed identity. Il `jobName` è il `requestId`, quindi **un replay del messaggio non crea un secondo job**.

I **webhook non sono supportati**: il token vive nell'URL (quindi finisce nei log e negli script client), non sono idempotenti e non restituiscono lo stato del job. I tre runbook in `runbooks/` sono stati adattati per accettare parametri nominali, eliminando il ramo `$WebhookData` dell'originale.

Routing table (RUNBOOK_MAP)

```
{
  "Windows": {
    "runbook": "RBK-WindowsDisposal",
    "parameters": {
      "SerialNumbers": "$.device.serialNumber",
      "RequestId": "$.requestId",
      "Scenario": "$.scenario",
      "DryRun": "$.options.dryRun"
    },
    "timeoutMinutes": 20
  },
  "Apple": {
    "runbook": "RBK-AppleDisposal",
    "parameters": {
      "SerialNumbers": "$.device.serialNumber",
      "MdmServerId": "$.options.mdmServerId",
      "RequestId": "$.requestId",
      "Scenario": "$.scenario",
      "DryRun": "$.options.dryRun"
    },
    "timeoutMinutes": 45
  },
  "Android": {
    "runbook": "RBK-AndroidDisposal",
    "parameters": {
      "Serials": "$.device.serialNumber",
      "RequestId": "$.requestId",
      "Scenario": "$.scenario",
      "DryRun": "$.options.dryRun"
    },
    "timeoutMinutes": 20
  }
}
```

I valori `$.a.b` sono percorsi JSON valutati sul messaggio canonico: aggiungere una piattaforma (es. Android Zero-Touch) è una modifica di configurazione, non di codice.

Segreti: Automation Variables

I runbook **non accettano segreti come parametri**: i parametri di un job Automation sono memorizzati in chiaro nei metadati del job e sono leggibili da chiunque abbia il ruolo Job Reader. Ogni credenziale viene letta da una Automation Variable tramite `Get-RequiredAutomationVariable`, che fallisce con un messaggio esplicito se la variabile manca o è vuota.

Il template crea le variabili; quelle segrete sono create **vuote e cifrate** e vanno popolate dopo il deploy (portale o `az automation variable update`).

Variabile	Cifrata	Usata da	Contenuto
<code>ClientId</code>	no	tutti	App registration (application) ID per Graph
<code>TenantId</code>	no	tutti	Tenant ID Entra
<code>GraphCertificateName</code>	no	tutti	Nome dell' Automation Certificate asset con il certificato Graph app-only (default <code>GraphAppCert</code>)
<code>Certificate_thumbprint</code>	sì	tutti	Thumbprint del certificato app-only (validazione + fallback su cert store)
<code>ClientSecret</code>	sì	tutti	Client secret dell'app registration. Fallback opzionale : usato dai runbook solo se non è disponibile alcun certificato
<code>ABM-ClientId</code>	sì	Apple	Client ID API Apple Business Manager
<code>ABM-KeyId</code>	sì	Apple	Key ID API ABM
<code>ABM-PrivateKey</code>	sì	Apple	Chiave privata EC P-256 in PEM
<code>KME-ClientIdentifier</code>	sì	Android	Client identifier Knox
<code>KME-KeysJson</code>	sì	Android	Keys JSON Knox (contiene la chiave RSA privata)
<code>KME-CustomerId</code>	sì	Android	Customer ID Knox

Autenticazione Graph app-only (runbook). I runbook si autenticano a Microsoft Graph con questo **ordine di preferenza**: (1) **certificato** dall'Automation Certificate asset (default GraphAppCert, recuperato con `Get-AutomationCertificate` e passato a `Connect-MgGraph -Certificate`); (2) certificato già presente nel cert store del sandbox, referenziato via `Certificate_thumbprint`; (3) **client secret** dalla variabile cifrata `ClientSecret`, usato **solo come ultima risorsa** se non è disponibile alcun certificato. Il certificato è l'opzione raccomandata. Esempio di caricamento del certificato:

```
az automation certificate create `
  --resource-group ASSET-TERMINATOR-DISPATCH-RG `
  --automation-account-name attdisp-auto-dev `
  --name GraphAppCert `
  --path .\graph-app.pfx `
  --password '<pfx-password>'
```

Deploy

```
cd poc-powershell-dispatch/infra

./deploy.ps1 `
  -ResourceGroup ASSET-TERMINATOR-DISPATCH-RG `
  -Subscription <subscription-id> `
  -Location westeurope `
  -GraphTenantId <tenant> `
  -GraphClientId <appId> `
  -GraphClientSecret <secret>
```

Lo script provisiona l'infrastruttura, sincronizza i moduli condivisi (`build.ps1`) e pubblica **entrambe** le Function App.

L'app registration Graph serve solo per le letture dell'intake:

- `DeviceManagementManagedDevices.Read.All` (application, con consenso admin)

I permessi di wipe restano sull'app registration usata dai runbook, non su questa.

Nota: `func azure functionapp publish` reimposta la subscription di default di `az`. Ogni comando `az` successivo deve passare `--subscription` esplicito (`deploy.ps1` lo fa già).

Connettività privata

Come per il PoC mock, la subscription applica una policy che forza `publicNetworkAccess=Disabled` su storage e Service Bus. Il template crea quindi VNet, subnet delegata, private endpoint e private DNS zone. Per un ambiente senza quella policy si può disattivare con `usePrivateEndpoints=false`.

Import dei runbook

Il template crea i tre runbook **vuoti**; `deploy.ps1` ne carica il contenuto da `runbooks/` e li pubblica, così una modifica al codice del runbook non è una modifica al template. Non serve nessun passo manuale.

Dopo il deploy vanno popolate le Automation Variable cifrate (vedi la tabella "Segreti: Automation Variables") e va caricato in Automation il certificato usato per l'autenticazione app-only a Graph.

Runbook: cosa è stato corretto

I runbook in `runbooks/` derivano da quelli del cliente, con questi fix:

Runbook	Problema originale	Gravità
Windows	<code>:IsNullOrWhiteSpace(...)</code> invece di <code>[string]::IsNullOrWhiteSpace(...)</code> (3x)	bloccante
Windows	assegnazione a <code>\$matches</code> , variabile automatica	alta
Windows	<code>"Serial=\$serial: ..."</code> interpretato come scope reference	bloccante
Apple	header <code>Authorization = "*****"</code> letterale: ogni chiamata ABM va in 401	bloccante
Apple	esito dell'activity di unassign mai verificato	alta
Apple	<code>Start-Sleep</code> cieco di 15 minuti in attesa del sync DEP	media
Android	accettava solo <code>\$WebhookData</code> , <code>throw</code> senza	bloccante
Android	<code>Escape-ODataString</code> : verbo non approvato e funzione inesistente	media
Android	<code>Get-Date -UFormat %s</code> per lo <code>iat</code> del JWT: dipende dal locale	media
Android	errori di wipe emessi con <code>Write-Error</code> : il job restava <code>Completed</code>	alta
tutti	nessun output strutturato per ServiceNow	alta
tutti	segreti passati come parametri o hardcoded	alta

Contratto comune dei tre runbook:

- parametri `-RequestId`, `-Scenario` (`Retirement` / `Sale` / `Disposal` / `LostStolen`), `-DryRun`, più i parametri di piattaforma;
- `-DryRun` è una **stringa**, non un `[bool]`: Automation passa i parametri come stringhe e `[bool]"false"` vale `$true`;
- tutti i segreti letti dalle Automation Variables, mai dai parametri;

- riga finale `##RESULT## {json}` con `requestId`, `platform`, `scenario`, `dryRun`, `wipeIssued`, `devices[]`, `errors[]`, `startedAt`, `completedAt`, emessa **anche sui percorsi di errore**;
- `throw` finale se `wipeIssued = $false`, così lo stato del job Automation riflette l'esito di business;
- le fasi di deregistrazione (ABM unassign, Knox delete) sono best-effort: un disservizio di Apple o Samsung non impedisce il wipe Intune.

Gli script client del cliente che invocano i webhook contengono i **token in chiaro nell'URL**: vanno revocati e rigenerati, e non vanno versionati.

Struttura

```
poc-powershell-dispatch/
├─ build.ps1          sincronizza shared/Modules → api/ e worker/
├─ shared/Modules/    AT.Common, AT.Graph, AT.State, AT.Messaging,
│                     AT.Automation, AT.Dispatch
├─ api/              WipeIntake (POST), GetStatus (GET)
├─ worker/           DispatchWindows/Apple/Android, JobMonitor
├─ runbooks/         i tre runbook di piattaforma corretti
├─ infra/main.bicep   infrastruttura completa
├─ infra/deploy.ps1   provisioning + publish
└─ samples/          payload di esempio
```

`api/Modules` e `worker/Modules` sono generate da `build.ps1`: non modificarle a mano, la sorgente è `shared/Modules`.